

SparseGrad: A Selective Method for Efficient Fine-tuning of MLP Layers

Yiktoriiia Chekalina^{1,2} Anna Rudenko^{1,2} Gleb Mezentshev^{1,2} Alexander Mikhalev² Alexander Panchenko^{2,1}

¹Artificial Intelligence Research Institute

²Skolkovo Institute of Science and Technology

March 27, 2026

Abstract

The performance of Transformer models has been enhanced by increasing the number of parameters and the length of the processed text. Consequently, fine-tuning the entire model becomes a memory-intensive process. High-performance methods for parameter-efficient fine-tuning (PEFT) typically work with Attention blocks and often overlook MLP blocks, which contain about half of the model parameters. We propose a new selective PEFT method, namely SparseGrad, that performs well on MLP blocks. We transfer layer gradients to a space where only about 1% of the layer’s elements remain significant. By converting gradients into a sparse structure, we reduce the number of updated parameters. We apply SparseGrad to fine-tune BERT and RoBERTa for the NLU task and LLaMa-2 for the Question-Answering task. In these experiments, with identical memory requirements, our method outperforms LoRA and MeProp, robust popular state-of-the-art PEFT approaches.

1 Introduction

Due to the tendency to increase the size of transformer models with each new generation, we need efficient ways to fine-tune such models on downstream task data. The usual practice is fine-tuning a large pre-trained foundational model on a downstream task. The major problem that prevents efficient fine-tuning is a steady increase in the memory footprint. One of the best strategies is high-performance methods for parameter-efficient fine-tuning (PEFT). Typically, such methods as LoRA focus on attention blocks and do not consider dense MLP blocks. Since MLP blocks can take a significant fraction of the model parameters (see Table 1), we propose to focus instead on MLP blocks. We introduce a novel selective PEFT approach called SparseGrad. Our method is based on finding a special sparsification transformation that allows us to fine-tune about 7% of the dense MLP layer parameters and still show good performance in downstream tasks.

We validate our approach on BERT and RoBERTa mod-

Blocks, Model	BERT	RoBERTa _{base}	RoBERTa _{large}	LLaMa-2
Full model	109M	100%	355M	100%
6.7B	100%			
MLP	54M	50%	178M	50%
3.4B	51%			
Embeddings	24M	22%	50M	14%
0.8B	12%			
Attention	25M	23%	116M	33%
2.4B	36%			
LayerNorm	1M	1%	4M	1%
0.1B	1%			

Table 1: Number of parameters for different layers in models based on the Transformer.

els on GLUE benchmark and in both cases obtain results better than LoRA and MeProp methods. We also fine-tune LLaMa-2 7B on the OpenAssistant dataset and also achieve performance higher than LoRA and MeProp.

2 Related Work

In the last few years, many approaches to PEFT have appeared. This distinguishes three types of methods: additive, reparametrization-based, and selective. In additive PEFT, small neural networks called adapters are added to the main model to steer the outputs of its modules. Adapters are trainable, therefore, the main model remains unchanged. We adapt this approach to NLP. In reparametrization-based approaches low-rank representations of trainable parameters are used. For example, LoRA parameterizes the weight update by a trainable low-rank matrix decomposition. In the original paper, LoRA is applied to self-attention modules, but not to MLP ones. In the selective methods, parts of the model or sets of the parameters are chosen for fine-tuning using some heuristics. Such methods include, for example, BitFit or MeProp, where only top-k parameters are updated during backpropagation. The approach proposed in this paper is related to selective methods.

3 Method

Our aim is to reduce the amount of trainable parameters at the fine-tuning stage. Taking into account that fine-tuning data is restricted to a limited scope, we assume there is a basis where the weight gradient matrix is very close to being sparse. To identify this basis, we applied a decomposition technique to the stacked weight gradient matrices. As a result, we introduce a new PyTorch layer class, `SparseGradLinear`, which transitions weights to this sparse gradient space, accumulates gradients in sparse form, and enables the reverse transition back to the original space.

3.1 Preliminary Phase: Finding Transition Matrices

To obtain transition matrices, an initial procedure is necessary. During this, we perform n_{steps} steps of standard backpropagation by freezing the entire model and unfreezing only the linear layers in MLP blocks. We do it to obtain the set of weights gradient matrices $\frac{\partial \mathcal{L}}{\partial W} \in \mathbb{R}^{D_{\text{in}} \times D_{\text{out}}}$. Stacking these matrices over n_{blocks} —the number of all blocks in the model—and over n_{steps} , we obtain a 3D tensor of size $D_{\text{in}} \times D_{\text{out}} \times (n_{\text{steps}} \cdot n_{\text{blocks}})$.

Applying Higher Order SVD (HOSVD) ? to this tensor yields matrices $U \in \mathbb{R}^{D_{\text{in}} \times D_{\text{in}}}$, corresponding to the dimension D_{in} and $V \in \mathbb{R}^{D_{\text{out}} \times D_{\text{out}}}$, corresponding to D_{out} . In this way, we get two orthogonal transition matrices U, V which are shared across all blocks of the model. Multiplying the layer’s weight matrix on the left by U and on the right by V^T transforms it into a new space. In this transformed space, the gradient matrix exhibits greater sparsity compared to the original space. Examples of $\frac{\partial \mathcal{L}}{\partial W}$ with and without transition to the new space are shown in Fig. 1.



Figure 1: Gradients on the 5-th BERT MLP: $U^T \frac{\partial \mathcal{L}}{\partial W} V$ (right) is more sparse than the original $\frac{\partial \mathcal{L}}{\partial W}$ (left).

3.2 Signal Propagation in SparseGradLinear Layer

Given a Transformer Linear layer with a weight matrix W , input activation X , and output $Y = XW^T$, we define the gradients of the output, input, and weights as $\frac{\partial \mathcal{L}}{\partial Y}$,

$\frac{\partial \mathcal{L}}{\partial X}$, and $\frac{\partial \mathcal{L}}{\partial W}$, respectively. To create the corresponding `SparseGradLinear` layer, we represent the weights in the U, V basis, such that the new weights are $W' = U^T W V$. Since the modules following `SparseGradLinear` remain unchanged in both forward and backward passes, it is crucial to maintain consistency between outputs of the Original Linear Layer Y and the `SparseGradLinear` layer $\hat{Y}$, as well as their input gradients $\frac{\partial \mathcal{L}}{\partial X}$ and $\frac{\partial \mathcal{L}}{\partial \hat{X}}$.

Table 2 outlines these adjustments and illustrates the correspondence of variables in Torch Autograd for Linear and `SparseGradLinear` layers.

Variable/Layer	Linear	SparseGrad
Weights	W	$W' = U^T W V$
Input	X	$\hat{X} = XU$
Output	$Y = XW^T$	$\hat{Y} = \hat{X}W'^T V^T$
Grad Output	$\frac{\partial \mathcal{L}}{\partial Y}$	$\frac{\partial \mathcal{L}}{\partial \hat{Y}} = \frac{\partial \mathcal{L}}{\partial Y} V$
Grad Input	$\frac{\partial \mathcal{L}}{\partial X} = \frac{\partial \mathcal{L}}{\partial Y} W$	$\frac{\partial \mathcal{L}}{\partial \hat{X}} = \frac{\partial \mathcal{L}}{\partial \hat{Y}} W'$
Grad Weights	$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial Y}^T X$	$\frac{\partial \mathcal{L}}{\partial W'} = \frac{\partial \mathcal{L}}{\partial \hat{Y}}^T \hat{X}$

Table 2: Correspondence of variables in Torch Autograd for a regular Linear layer and `SparseGradLinear`.

Thus, `SparseGradLinear` is equivalent to 3 linear layers: first with frozen weights U^T , defined by the HOSVD, second with trainable new weights $W' = U^T W V$, third with frozen weights V , defined by the HOSVD. Fig. 2 shows the propagation of the signal in this structure.

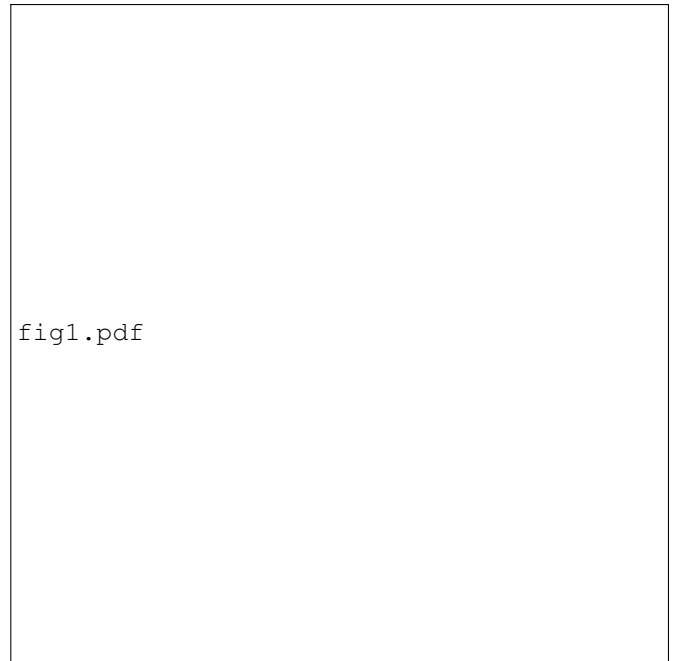


Figure 2: The first row illustrates signal propagation in the original Linear Layer, while the second row illustrates propagation with the proposed `SparseGradLinear` layer.

3.3 Sparse-by-Dense Matrix Multiplication

We provide the `SparseGradLinear` class with updated Forward and Backward procedures. However, the addition of multiplications by U, V into them increased the execution time and affected peak memory in the training loop.

The sparsity of the gradient tensor $\frac{\partial \mathcal{L}}{\partial W'} = \frac{\partial \mathcal{L}}{\partial Y}^T \hat{X}$ results in some of the multipliers being sparse. We explore the structure of each component in this formula and figure out that $\frac{\partial \mathcal{L}}{\partial W'}$ has a sparsity approximately equal to 99%. Histograms of the percent of its non-zero elements are presented in Fig. 3. It also shows that the sparsity is “strided”—most of the rows are completely filled with zeros. These rows can be excluded from the multiplication procedure, thus optimizing it.



Figure 3: Strided structure of $\frac{\partial \mathcal{L}}{\partial W'}$ (left) and visualizations of % nonzero elements in $\frac{\partial \mathcal{L}}{\partial W'}$ throughout training (right).

More precisely, to multiply the sparse matrix $A \in \mathbb{R}^{b \times c}$ by a dense matrix $B \in \mathbb{R}^{c \times d}$ we select `rows` and `cols`—indices of rows and columns of A which contain nonzero elements and multiply as follows:

$$C = A(\text{rows}, :)(:, \text{cols})B(\text{cols}, :). \quad (1)$$

We employ C either for further multiplications, or convert it into COO format and send it to SparseAdam optimizer. Indexes in COO format are defined by restoring indexes of A :

$$C_{\text{coo}}(\text{rows}(k), \text{cols}(l)) = C(k, l). \quad (2)$$

As it is shown in Table 3, such procedure significantly speeds up the harnessing of `SparseGradLinear`.

4 Time and Memory Consumption per Training Iteration

We measure the peak memory allocated during training using the CUDA memory allocator statistics. Table 3 demonstrates this statistic on average for all GLUE datasets for the RoBERTa_{base} model. The comprehensive Tables 7 and 8, which outline metrics for each dataset separately, can be found in Appendix A. Among all methods, LoRA presents the most efficient memory usage, preserving 30% of the peak memory.

SparseGrad, while using slightly more memory, still achieves a 20% savings. The increase in peak memory with SparseGrad is attributed to the maintenance of matrices U and V and their multiplication by the dense objects, such as Input X .

Method	Steps/Sec.	Memory, MB
Regular FT	4.11	1345
LoRA	14.7	944
SparseGrad (SD)	4.3	1100
SparseGrad (Reg)	0.9	1210

Table 3: Training speed and memory requirements averaged on the GLUE benchmark. The last two rows report the results for the SparseGrad method with Sparse-by-Dense (SD) and Regular (Reg) matrix multiplication, respectively.

In terms of training time, LoRA demonstrates the fastest training, followed by SparseGrad, and then standard fine-tuning. Table 3 shows that Sparse-by-Dense multiplication saves approximately 12% memory leading to an almost five-fold increase in speed.

5 Experiments

We conducted experiments on three transformer-based encoder models, BERT and RoBERTa base and large, on the GLUE ? benchmark, and the LLaMa-2 decoder model on the OpenAssistant Conversations corpus ?. We compared the fine-tuning of the full model (Regular FT scheme) with three PEFT methods, namely LoRA, MeProp and SparseGrad, applied to MLP blocks. To harness LoRA, we use an official repository code. For the MeProp method, we kept the largest elements in the $\frac{\partial \mathcal{L}}{\partial W}$ matrix. The proposed SparseGrad involves replacing layers in MLP blocks with its `SparseGradLinear` equivalents.

5.1 Natural Language Understanding with BERT and RoBERTa

We explore the acceptable sparsity level of the gradient matrices in the “sparse” space, $\frac{\partial \mathcal{L}}{\partial W'}$, by varying the number of remaining parameters in the Linear Layer from 10^5 to 1.8×10^4 , we fine-tuned the model on the GLUE benchmark and identified the point at which performance begins to degrade. This occurs when the number of trainable parameters reaches 2.2×10^4 , corresponding to 7% of the total weights. Full experimental results can be found in Appendix C.

Guided by this heuristic, in our experiments we leave the top 1% of the largest elements and set the rest to zero. To deal with SparseGradients, we use the SparseAdam optimizer—the masked version of the Adam algorithm. The remaining model parameters are trained with the standard AdamW optimizer.

We fine-tune BERT, RoBERTa_{base} and RoBERTa_{large} ? using Regular FT, LoRA, MeProp and SparseGrad schemes for 20 epochs with early stopping for each task in the GLUE. We varied the batch size and learning rate using the Optuna framework ?. The learning rate ranged from 10^{-6} to 10^{-1} , and the batch size is selected from the set $\{8, 16, 32\}$. Optimal training parameters for each task are available in the Appendix D. In LoRA we take the rank 10 for RoBERTa_{large} and rank 7 for BERT and RoBERTa_{base}. For SparseGrad and MeProp we keep the same number of parameters—approximately 1% of each Linear layer.

The average scores for all GLUE tasks for BERT and RoBERTa_{base} are in Table 4; per-task results are placed in the Appendix B. Table 5 depicts the scores for the RoBERTa_{large} model.

Our results indicate that SparseGrad outperforms LoRA with an equivalent number of trainable parameters across all models. For BERT, SparseGrad even exceeds the performance of Regular FT. This may be attributed to the changing basis of the weights in SparseGrad acting as a form of regularization. Concerning MeProp, it provides weaker results than SparseGrad in all cases except the RoBERTa_{large} on CoLA. This could be explained by the fact that our approach first transforms the elements into a special “sparse” space, while MeProp operates on gradients in the original space. In the original space, the histogram of elements is flatter (see Fig. 1), which suggests that, with the same cut-off threshold, MeProp may remove more significant elements compared to SparseGrad.

Model	BERT	RoBERTa _{base}
Regular FT	82.5	84.2
LoRA	81.0	83.1
SparseGrad	82.0	83.8
MeProp	82.1	82.5

Table 4: Average scores over the GLUE benchmark for BERT and RoBERTa_{base} models.

5.2 Conversations with LLaMa-2

We apply the SparseGrad method to fine-tune LLaMa-2 7B ? model on the OpenAssistant conversational dataset ?. Fine-tuning was performed on a single GPU NVIDIA A40 during 1 epoch with learning rate 9×10^{-4} . For Regular FT, we unfroze `up_proj` and `down_proj` layers in the MLP modules with a block index divisible by 3 (0, 3, 6, ...). We apply LoRA with rank 32 to the selected blocks, leaving the rest of the model untrainable. In the SparseGrad and MeProp methods, we also consider selected MLP modules in the transformer and leave = 100,000 (0.27%) nonzero elements in the gradient matrix. For LLaMa-2, we conducted a similar ablation study as we did for BERT and RoBERTa. We varied the number of remaining

parameters in the MLP block and identified the point where the model’s performance began to decline.

We validate obtained models on the question set MT-Bench Inf from Inflection-Benchmarks ?. We followed the guidelines outlined in this work, called “Single Protocol” or “Single Answer Grading”. We got the answers by using the FastChat platform¹ and then evaluating them using GPT-4. GPT-4 rates the answers on a scale of 1 to 10, with the evaluation prompt taken from ?.

The resulting losses and average GPT-4 scores are presented in Table 6. While the models perform similarly overall, SparseGrad slightly outperforms LoRA, MeProp, and regular fine-tuning. Examples of responses to Inflection-Benchmark samples are provided in Appendix E. These examples illustrate that, although all models produce good answers, the LoRA-trained model occasionally overlooks important nuances. In the examples given, it fails to recognize that presentations can be stressful for introverts or that hierarchy plays a significant role in Japanese corporate culture.

6 Conclusion

We propose a new selective PEFT method called SparseGrad, which identifies a space where the gradients exhibit a sparse structure and updates only its significant part. SparseGrad is validated through experiments conducted on the BERT, RoBERTa and LLaMa-2 models, demonstrating its superiority over the additive LoRA and selective MeProp methods.

Leveraging the sparsity property significantly accelerated the calculations in SparseGrad. Our method runs faster than standard fine-tuning but slower than LoRA, while yielding better performance than LoRA; the same trend applies to memory usage. In summary, our method serves as an alternative to LoRA in situations where the performance of the final model takes precedence over the execution time. The source code as well as links to pretrained models are available at repository.²

7 Acknowledgements

The work was supported by the Analytical center under the RF Government (subsidy agreement 000000D730321P5Q0002, Grant No. 70-2021-00145 02.11.2021).

8 Limitations

The main limitation of our method is the additional memory requirements during the Preliminary Phase. The extra memory is assessed as follows: we need to unfreeze the MLP layers, which hold approximately half of the training parameters in Transformers (see Table 1), store and decompose a large

¹<https://github.com/lm-sys/FastChat>

²https://github.com/sayankotor/sparse_grads

Method	#Trainable params	STS-B	CoLA	MNLI	MRPC	QNLI	QQP	RTE	SST-2
FT Regular	355M	91.9±0.4	67.1±2.6	90.8±0.2	89.9±0.3	92.9±0.5	91.8±0.1	63.9±2.0	94.1±0.1
LoRA	16.8M	92.1±0.1	64.4±0.8	90.7±0.2	89.9±0.3	93.2±0.3	91.8±0.2	60.2±4.1	96.6±0.1
SparseGrad	0.05M	92.4±0.1	63.2±1.1	90.7±0.1	90.4±0.2	93.3±0.5	91.1±0.1	64.7±6.1	96.8±0.1
MeProp	0.05M	92.0±0.1	63.7±1.1	90.4±0.2	89.8±0.5	92.5±0.3	91.4±0.1	59.2±2.4	96.2±0.1

Table 5: Comparative results of RoBERTa_{large} for 20-epoch task-specific fine-tuning.

Method	#Train params	Valid Loss	MT-Bench Score
Regular FT	22%	1.250±0.05	4.407
LoRA	0.5%	1.249±0.04	5.025
SparseGrad	0.5%	1.222±0.03	5.132
MeProp	0.5%	1.239±0.04	4.261

Table 6: Comparative results for LLaMa-2 on the OpenAssistant dataset.

tensor. For instance, 30 steps in the preliminary phase result in a tensor of approximately 276 MB for BERT and RoBERTa models, and 5.2 GB for LLaMa-2 7B models. The decomposition part can be the most memory-consuming, as it involves reshaping a 3-dimensional tensor into a matrix with a dimension size equal to the product of two dimension sizes of the tensor ?.

However, this part is executed only once during the entire fine-tuning process and can be computed on the CPU in a short time. The Higher Order SVD decomposition of such objects takes approximately 78 seconds for BERT and RoBERTa_{base} layers and about 668 seconds for LLaMa on an Intel Xeon Gold 6342 CPU processor.

9 Ethics Statement

Our proposed approach involves a novel method for fine-tuning large language models, which can be considered as cost-effective as we only update 0.1% of the weights. This type of fine-tuning is environmentally friendly as it reduces resource wastage. We utilized pre-trained models from the Hugging Face repository and implemented updates using the Pytorch library. We exclusively used open-source datasets to avoid any potential harm or ethical concerns. By prioritizing ethical standards and recognizing potential risks, we strive to promote responsible and sustainable research practices.

Method/Dataset	STS-B	CoLA	MNLI	MRPC	QNLI	QQP	RTE
Regular FT	4.11	2.9	4.3	4.2	4.1	3.1	4.7
LoRA	14.4	14.6	14.7	14.8	14.6	14.3	14.3
SparseGrad (SD)	4.1	4.3	4.3	6.3	6.2	5.8	4.4
SparseGrad (Reg)	0.9	0.3	0.4	3.1	3.9	1.8	1.5

Table 7: The training step execution speed, measured in steps per second (where a higher value indicates faster execution), is reported for the RoBERTa base model. The last two rows describe the SparseGrad method with Sparse-by-Dense multiplication and with Regular matrix multiplication.

Method/Dataset	STS-B	CoLA	MNLI	MRPC	QNLI	QQP	RTE
Regular FT	1345	1344	1358	1350	1362	1369	1333
LoRA	944	969	978	986	998	938	933
SparseGrad (SD)	1000	1021	1017	1101	1101	1010	1083
SparseGrad (Reg)	1210	1283	1272	1256	1183	1245	1173

Table 8: Peak memory measurement in MB for training loop for the model RoBERTa base.

A Appendix A

B Appendix B

C Appendix C

The average GLUE results for the BERT and RoBERTa_{base} models with respect to the number of remaining updated parameters in Linear layers. Tables 11 and 12 show that under the 0.8% of the remaining parameters, performance tends to decrease.

D Appendix D

Best training parameters for all models. In all experiments, we repeat fine-tuning 3 times over different seeds and report the average score.

Method SST-2	#Trainable params	AVG	STS-B	CoLA	MNLI	MRPC	QNLI	QQP	RTE
Regular FT 92.7±0.4	109M	82.5	89.3±0.4	59.0±4.8	84.0±0.6	86.2±1.2	91.1±0.9	91.3±0.1	61.4±4.8
LoRA 92.4±0.4	53M	81.6	89.3±0.4	58.6±1.2	84.0±0.4	86.2±1.1	90.9±0.4	91.3±0.1	58.4±3.0
SparseGrad 92.3±0.1	0.03M	82.6	89.4±0.3	64.8±1.2	84.1±0.4	86.6±0.5	90.4±0.4	91.5±0.1	58.8±0.8
MeProp 92.4±0.1	0.03M	82.1	88.9±0.4	64.6±1.1	83.8±0.6	86.2±0.8	90.4±0.4	91.4±0.1	58.4±0.8

Table 9: Comparative results of BERT model for 20-epoch task-specific fine-tuning.

Method SST-2	#Trainable params	AVG	STS-B	CoLA	MNLI	MRPC	QNLI	QQP	RTE
Regular FT 94.7±0.1	125M	84.2	90.4±0.4	59.7±1.0	87.7±0.3	90.0±0.0	90.6±0.3	91.5±0.1	68.8±2.5
LoRA 94.5±0.1	68M	83.1	90.5±0.1	60.6±1.4	87.4±0.3	89.4±0.5	91.5±0.1	91.5±0.1	63.1±1.4
SparseGrad 94.2±0.1	0.03M	83.6	90.8±0.1	60.0±1.0	87.6±0.3	89.8±0.4	91.0±0.2	91.3±0.1	65.6±1.4
MeProp 94.2±0.1	0.03M	82.5	90.5±0.2	59.2±1.4	87.0±0.4	89.7±0.3	90.6±0.2	91.4±0.1	61.5±1.0

Table 10: Comparative results of RoBERTa_{base} for 20-epoch task-specific fine-tuning.

Method SST-2	% #remained in Linear Layers	AVG	STS-B	CoLA	Method SST-2	% #remained in Linear Layers	AVG	STS-B	CoLA	Method SST-2	% #remained in Linear Layers	AVG	STS-B	CoLA	MNLI
SparseGrad 100k 92.4±0.1	100	82.0	89.2±0.4	58.8±0.8	SparseGrad 100k 94.2±0.1	100	86.6±0.4	89.4±0.3	80.0±0.0	SparseGrad 100k 94.2±0.1	100	86.6±0.4	89.4±0.3	80.0±0.0	87.5±0.0
SparseGrad 18k 92.4±0.1	0.8	81.5	89.1±0.3	59.1±0.8	SparseGrad 18k 94.0±0.1	0.8	84.6±0.3	88.8±0.4	80.4±0.0	SparseGrad 18k 94.0±0.1	0.8	84.6±0.3	88.8±0.4	80.4±0.0	87.4±0.0
SparseGrad 22k 92.3±0.1	1.0	82.2	89.7±0.0	60.0±0.0	SparseGrad 22k 94.0±0.1	1.0	84.6±0.3	89.3±0.3	81.8±0.0	SparseGrad 22k 94.0±0.1	1.0	84.6±0.3	89.3±0.3	81.8±0.0	87.7±0.0
SparseGrad 30k 92.4±0.1	1.2	82.0	89.2±0.2	59.1±0.4	SparseGrad 30k 94.2±0.1	1.2	85.4±0.0	89.3±0.3	80.8±0.0	SparseGrad 30k 94.2±0.1	1.2	85.4±0.0	89.3±0.3	80.8±0.0	87.6±0.0
SparseGrad 100k 92.4±0.1	4.2	82.2	89.3±0.0	60.0±0.4	SparseGrad 100k 94.1±0.1	4.2	85.1±0.3	88.9±0.4	82.0±0.0	SparseGrad 100k 94.1±0.1	4.2	85.1±0.3	88.9±0.4	82.0±0.0	87.0±0.0

Table 11: GLUE score as a function of the weight gradient sparsity in BERT.

Table 12: GLUE score as a function of the weight gradient sparsity in RoBERTa_{base}.

E Appendix E

Responses from the models to an example from Inflection-Benchmarks are shown. While all models perform fairly well, the LoRA-trained model overlooks the fact that public speaking can be stressful for an introvert when answering the first question.

[Question 1]

Write a persuasive email to convince your introverted friend,

who dislikes public speaking, to volunteer as a guest speaker at a local event. Use compelling arguments and address potential objections. Please be concise.

LLaMa2-7B Regular FT:

Here’s a persuasive email to convince your introverted friend to volunteer as a guest speaker at a local event: Subject: Join us as a guest speaker at our local event

Dear (Friend’s Name), I hope this email finds you well. I hope you’re doing well. I’m writing to invite you to be a guest

Dataset	batch size	learning rate
STS-B	32	1.24e-4
CoLA	32	3.15e-5
MNLI	32	6.07e-6
MRPC	32	1.22e-5
QNLI	16	1.47e-5
QQP	16	6.51e-5
RTE	16	1.04e-4
SST-2	32	1.02e-5

Table 13: Best training parameters on GLUE benchmark for BERT model.

Dataset	batch size	learning rate
STS-B	16	2.10e-5
CoLA	16	1.01e-5
MNLI	32	1.51e-5
MRPC	32	1.09e-5
QNLI	16	1.91e-5
QQP	16	5.11e-6
RTE	16	3.05e-5
SST-2	16	1.33e-5

Table 14: Best training parameters on GLUE benchmark for RoBERTa_{base} model.

speaker at our upcoming local event. As a friend, I know how passionate you are!

LLaMa2-7B LoRA:

Subject Line: Please, Volunteer as a Guest Speaker! Dear John, I hope you are doing well. I am writing to you because I know you are an introverted person and do not like public speaking. However, I have a great opportunity for you to volunteer as a guest speaker at the event!

LLaMa2-7B SparseGrad:

Subject: Volunteer as a Guest Speaker at a Local Event
Dear [Name], I hope this email finds you well. I wanted to reach out to you about an exciting opportunity to volunteer as a guest speaker at a local event. As you know, public speaking can be a nerve-wracking experience, but it's also a great way to share your knowledge and expertise with others.

[Question 2]

What are some business etiquette norms when doing business in Japan?

LLaMa2-7B Regular FT:

If you're doing business in Japan, it's important to follow their business etiquette norms. Here are some key things to keep in mind:

1. Hierarchy: In Japan, it's important to respect the hierarchy in the workplace. Address your Japanese colleagues with the appropriate title, and don't speak to them unless they initiate the conversation.
2. Punctuality: Being punctual is considered very important in

Dataset	batch size	learning rate
STS-B	32	1.41e-5
CoLA	16	1.80e-5
MNLI	16	1.15e-6
MRPC	32	2.47e-5
QNLI	16	8.83e-6
QQP	32	7.11e-6
RTE	32	1.02e-5
SST-2	32	1.02e-5

Table 15: Best training parameters on GLUE benchmark for RoBERTa_{large} model.

Japanese culture, so make sure to arrive on time for meetings and events.

3. Business Cards: Business cards are an important part of doing business in Japan. Make sure to have your own business card made beforehand and carry it with you at all times.

LLaMa2-7B LoRA:

Of course! Here are some business etiquette norms when doing business in Japan:

1. Business Cards: Business cards are a vital part of business interactions in Japan. Always have a few business cards with you and exchange them with others during introductions.
2. Punctuality: Time is sacred in Japan, and being punctual is essential for business meetings. Arrive on time or even a few minutes early.
3. Gifts: Gifts are common in business interactions in Japan, especially for special occasions. Small personalized gifts such as chocolates or flowers are appropriate.

LLaMa2-7B SparseGrad:

Here are some business etiquette norms to keep in mind when doing business in Japan:

1. Respect for seniority: In Japanese culture, age and seniority are highly respected, so it is important to show respect to those who are older or in a higher position than you.
2. Business cards: Business cards are an important part of doing business in Japanese culture, and it is customary to exchange business cards at the beginning of a meeting.
3. Gifts: Gifts are often given as a sign of appreciation or gratitude in Japanese culture. It is important to give gifts that are appropriate for the occasion and not too extravagant.